



UNIVERSITY OF  
ABERDEEN

University of Aberdeen  
School of Natural and Computing Sciences  
Department of Computing Science

2024 – 2025

**Programming assignment – Individually Assessed (no teamwork)**

**Deadline: 22:00, 7<sup>th</sup> April 2025 (SCNU local time)**

**Title: JC1503 – Object-Oriented  
Programming**

Note: This assignment accounts for **30%** of your total mark of the course (Code and report will account for 20% and 80% of the whole assessment, respectively).

**Information for Plagiarism:** The source code and your report will be submitted for plagiarism check (e.g., *Turnitin* or other tools). Any AI tools which automatically help to edit, paraphrase, or generate your source code and report are not allowed. If those behaviours are detected, your submissions will be classified as plagiarism. Please refer to the slides available at *MyAberdeen* for more information about avoiding plagiarism before you start working on the assessment. Please also read the following information provided by the university:

<https://www.abdn.ac.uk/sls/online-resources/avoiding-plagiarism/>

## Introduction

This coursework involves designing and implementing a **Smart Home Automation System** using Python and Object-Oriented Programming principles. The system will manage and control smart devices within a home environment using OOP concepts such as inheritance, polymorphism, encapsulation, and abstraction. The project will involve implementing various components such as devices, device controllers, and a central home automation hub. It will also cover data structures like queues and maps, recursion for system tasks, exception handling for reliability, and design patterns like the Singleton pattern.

The Smart Home Automation System will allow users to manage smart devices like lights, thermostats, and security cameras. Each device will be represented as an object with distinct properties and methods. Devices will communicate with a central hub, which processes commands and stores device status.

System Features:

- Add/Remove Devices: Users can add or remove devices from the system.
- Command Devices: Control device operations like turning lights on/off or setting thermostat temperature.
- Monitor System State: View the current status of all devices.
- Energy Management: Monitor energy consumption.
- Scheduler: Schedule device operations based on time or events.

## Guidance and Requirements

For this assessment, you will be required to complete coding tasks and a final written report that outlines the steps you took. To help you get started, we will provide you with a template code and a template report. It is **important** that your assessment code and report **strictly follow** the designated structure and include all required content as outlined in each section. Marks are allocated for each subtask, and your written report should contain all distinct sections/subtasks that provide a comprehensive and reflective account of the processes you undertook.

To test your implementation in Codio, the project template includes five tests. Upon running the test.py file by command **python3 test.py**, you should expect all five tests to fail initially, indicating that your implementation is not yet correct.

```

FEFFF
=====
ERROR: test_controller_execute_command (__main__.TestSmartHomeSystem)
=====
Traceback (most recent call last):
  File "test.py", line 33, in test_controller_execute_command
    self.assertEqual(self.controller.devices['L1'].get_status(), 'on')
KeyError: 'L1'
=====
FAIL: test_controller_add_remove_device (__main__.TestSmartHomeSystem)
=====
Traceback (most recent call last):
  File "test.py", line 26, in test_controller_add_remove_device
    self.assertIn('L1', self.controller.devices)
AssertionError: 'L1' not found in {}
=====
FAIL: test_device_inheritance_and_methods (__main__.TestSmartHomeSystem)
=====
Traceback (most recent call last):
  File "test.py", line 48, in test_device_inheritance_and_methods
    self.assertEqual(self.light.get_status(), 'on')
AssertionError: None != 'on'
=====
FAIL: test_device_operations (__main__.TestSmartHomeSystem)
=====
Traceback (most recent call last):
  File "test.py", line 18, in test_device_operations
    self.assertEqual(self.light.get_status(), 'off')
AssertionError: None != 'off'
=====
FAIL: test_hub_schedule_task_and_energy_usage (__main__.TestSmartHomeSystem)
=====
Traceback (most recent call last):
  File "test.py", line 41, in test_hub_schedule_task_and_energy_usage
    self.assertEqual(total_energy, sum(device.get_energy_usage() for device in self.hub.controller.devices.values()))
AssertionError: None != 0
=====

Ran 5 tests in 0.001s

FAILED (failures=4, errors=1)

```

Once you have completed all the steps, after running **python3 test.py** again in Codio, you should see the following, if you have implemented the assignment correctly:

```

. Executed on on Living Room Light
. Executed off on Living Room Light
... Task scheduled: on L1 at 2025-02-18 14:20:57
.
-----
Ran 5 tests in 0.001s
OK

```

Note that not all tasks have tests defined. **Make sure you mention in the report which tasks you have completed.**

In the later stages, you can use the command **python3 main.py** check the console output.

A detailed description of each task is given below. To complete these steps, follow the instructions and ensure you use the provided template. All processes taken to run the final project must be clearly described. For submission instructions refer to the later sections.

**\*\*Please read all the information below carefully and finish them based on requirements\*\***

### **Step 1: Design and Implement the Device Class (30 points)**

The **Device** class will act as the base class for all smart devices, which begins by defining the Device class with essential attributes: `device_id`, `name`, `status`, and `energy_usage`. It uses Python's `abc` module to create an abstract base class to ensure subclasses implement necessary methods. In addition, it implements getter and setter methods for encapsulated attributes. Finally, we will extend the Device class to create subclasses like **Light**, **Thermostat**, and **Camera**. Each subclass should have specific attributes and behaviors.

#### 1.1) Constructor and Attributes:

In the `__init__()` method, we will have four private attributes, such as Device ID, device name, status (e.g., "on", "off"), and energy consumption. You should build the corresponding getter and setter methods based on those private variables.

#### 1.2) Methods:

Implement the `__str__()` method to print device information. The output format should follow: "Device: xxx, ID: xxx, Status: xxx, Energy Usage: xxxkWh" by replacing the xxx with specific values from those private variables.

Then, you should add methods like `turn_on()`, `turn_off()`, and `get_energy_usage()` to control device behavior.

#### 1.3) Inheritance:

Create subclasses for **Light**, **Thermostat**, and **Camera** with specialized methods, where the initial brightness for Light subclass should 100, the initial temperature for Thermostat subclass should be 22 and the initial resolution for Camera subclass should be 1080p.

### **Step 2: Implement Device Controller (30 points)**

The **DeviceController** class will manage all devices. It will initialise an empty dictionary to track devices using their unique IDs, and then implement methods for adding, removing and listing devices.

#### 2.1) Data Structure:

Use a dictionary to map device IDs to device objects. The keys should be unique, and the values should be device instances.

#### 2.2) Methods:

Implement `add_device()` to add devices to the dictionary. Implement `remove_device()` to delete devices by their ID. Create `list_devices()` to print all devices in a readable format. Implement `execute_command()` to send commands like "on" or "off" to devices.

### **Step 3: Implement Smart Home Hub (30 points)**

The **SmartHomeHub** class will act as the central controller. It will implement the Singleton pattern to ensure only one instance of the SmartHomeHub exists, and develop methods to schedule tasks, display status, and calculate energy usage, and use recursion to compute total energy usage.

### 3.1) Singleton Pattern:

Override the `__new__()` method to create a single instance.

### 3.2) Methods:

Implement `schedule_task()` to plan device actions for a given time.

Create `display_status()` to print all devices and their states.

### 3.3) Recursion:

Write a recursive method to compute the sum of energy usage across all devices.

## **Step 4: Demonstrate the Functionality by main function (10 points)**

Use `if __name__ == '__main__':` and create a **main function** that starts your program. The following functionality should be implemented:

4.1) Create an instance of the SmartHomeHub class

4.2) Create three instances of the Light, Thermostat and Camera classes, with different device ID and name as below.

- light: L1, Living Room Light
- thermostat: T1, Home Thermostat
- camera: C1, Front Door Camera

4.3) Add those created instances to the hub's controller.

4.3) Call hub to display the current status of different devices.

4.4) Use hub's controller to execute command: turn L1 on and turn T1 off.

4.5) Schedule a task for C1 to turn it on based on the current time using datetime library.

4.6) Use hub to print the total energy usage of all devices.

## **Step 5: Write the report (100 points)**

Your report should be structured according to the template provided below.

1. Introduction (5 points, 100-150 words)

Paragraph 1: Project Overview (2 points)

- Clearly explain the purpose of the Smart Home Automation System.
- Describe how it helps manage smart devices using Python OOP principles.

Paragraph 2: Learning Objectives (3 points)

- Outline the educational goals, including practical OOP applications and design pattern usage.
- Briefly introduce the key components: devices, controller, and hub.

## 2. Device Class Design (20 points, 200-250 words)

### Paragraph 1: Class Structure (5 points)

- Explain the purpose of the Device base class and its attributes.
- Discuss encapsulation with private variables and getter/setter methods.

### Paragraph 2: Method Implementations (10 points)

- Detail methods like `turn_on()`, `turn_off()`, and `get_energy_usage()`.
- Provide code snippets illustrating method functionality.

### Paragraph 3: Inheritance and Subclasses (5 points)

- Describe how Light, Thermostat, and Camera inherit from the Device class.
- Highlight polymorphism through overridden or extended methods.

## 3. Device Controller Class Design (25 points, 300-350 words)

### Paragraph 1: Data Structure (5 points)

- Explain the use of dictionaries for device management.
- Justify why dictionaries are suitable for efficient lookup.

### Paragraph 2: Controller Methods (20 points)

- Discuss `add_device()`, `remove_device()`, `list_devices()`, and `execute_command()`.
- Provide examples of commands and expected system responses.

## 4. Smart Home Hub Class Design (25 points, 300-350 words)

### Paragraph 1: Singleton Pattern (5 points)

- Explain the need for a single SmartHomeHub instance.
- Show how the pattern ensures central control.

### Paragraph 2: Core Methods (15 points)

- Describe `schedule_task()`, `display_status()`, and `total_energy_usage()`.
- Include recursive logic explanation for energy calculation.

### Paragraph 3: Integration (5 points)

- Discuss how the hub communicates with the DeviceController.
- Provide insights into system-wide task coordination.

## 4. Testing and Demonstration (20 points, 200-250 words)

Paragraph 1: Testing Approach (5 points)

- Outline the use of unittest framework and edge case testing.
- Explain the role of automated tests in software reliability.

Paragraph 2: Test Implementation (10 points)

- Detail tests for Device, DeviceController, and SmartHomeHub.
- Include sample test cases with explanations.

Paragraph 3: Test Results (5 points)

- Present results, highlighting successful tests and any issues.

5. Conclusion (5 points, 100-150 words)

Paragraph 1: Summary (2 points)

- Summarize the project's goals and achievements.
- Reflect on the practical application of OOP concepts.

Paragraph 2: Learning Experience (2 points)

- Discuss challenges faced and lessons learned.

Paragraph 3: Future Improvements (1 point)

- Suggest enhancements for the Smart Home Automation System

**Useful Information**

- Please describe and justify each step that is needed to reproduce / run your work by using clear descriptive writing, supporting code-snippets and screenshots. When using screenshots please make sure they are clearly readable.
- If you use open source code, you must point out where it was obtained from (even if the sources are online tutorials or blogs) and detail any modifications you have made to it in your tasks. You should mention this in both your code and report. Failure to do so will result in zero marks being awarded on related (sub)tasks

**Marking Criteria**

- Quality of the source code, including the commenting of the code
- Technical correctness of the code, and structure.
- Reproducibility. How easy is it for the course coordinator to repeat your work based on your report and code?
- Quality of the report, including structure, clarity, and brevity

### **Submission Instructions**

You should submit a **PDF version** of your report along with your code. The name of the **PDF file** should have the form “JC1503\_<your Class programme>\_<your Surname>\_<your Firstname>\_<Your Aberdeen Student ID>”. For instance, “JC1503\_CS\_Smith\_John\_4568985.pdf”, where 4568985 is your Aberdeen student ID, and your class programme should be selected from AI, CS or BMIS.

**It is your responsibility to ensure that your code runs in Codio. If you do it on another device, and then move it to Codio, you need to ensure it runs there, and that it runs as expected.**

*You should submit your code and the report using Codio and MyAberdeen portal, respectively.*

Your Codio submission should **only** consist of **two files: main.py**, which contains your implementation, and **test.py**, which contains the test cases provided in the code template to validate the functionality of your implementation.

Any questions pertaining to any aspects of this assessment, please address them to one of the course coordinators.